

Основи роботи з базами даних

Тема 4

Зміст лекції

- Еволюція доступу до даних, JdbcTemplate та JdbcClient
- Концепція ORM та архітектура Jakarta Persistence
- Моделювання сутностей у Java 25
- Реляційні зв'язки та каскадні операції
- Стратегії завантаження та вирішення проблеми N+1
- Абстракція Spring Data JPA та інтерфейси репозиторіїв
- Кастомні запити JPQL та нативний SQL
- Робота з In-Memory базами даних H2

Еволюція доступу до даних та специфікація JDBC

Доступ до реляційних баз даних у Java традиційно базується на JDBC:

- Специфікація JDBC надає інтерфейс для виконання SQL-запитів через об'єкти `Connection`, `PreparedStatement` та `ResultSet`.
- Робота на рівні чистого JDBC вимагає значного обсягу шаблонного коду для відкриття з'єднань та контролю витоку ресурсів.

Перехід від чистого JDBC до Spring JdbcTemplate

Ручна робота з JDBC ускладнює розвиток та підтримку кодової бази:

- Необхідність обробки перевірюваних винятків `SQLException` перевантажує сервісний шар зайвими блоками перехоплення.
- Ручне зіставлення стовпців таблиці з полями класів збільшує ризик помилок при зміні схеми бази даних.
- Для усунення рутинних дій та збереження контролю над SQL фреймворк Spring пропонує компонент `JdbcTemplate`.

Шаблон проєктування Template Method у JdbcTemplate

Клас `JdbcTemplate` реалізує патерн Template Method:

- Фреймворк бере на себе незмінний життєвий цикл операції: взяття з'єднання з `DataSource`, створення виразу та повернення з'єднання в пул.
- Розробник визначає лише змінну частину логіки: текст SQL-запиту, вхідні аргументи та правило обробки результату.

Обробка винятків та безпека запитів у JdbcTemplate

Архітектура `JdbcTemplate` забезпечує надійність доступу до даних:

- Усі низькорівневі `SQLException` автоматично транслюються в ієрархію неперевірюваних винятків Spring `DataAccessException`.
- До них належать `BadSqlGrammarException`, `DuplicateKeyException` та `EmptyResultDataAccessException`.
- Параметризація запитів через символи `?` гарантує надійний захист від атак SQL Injection.

Відображення результатів вибірки через RowMapper

Для перетворення рядків `ResultSet` на Java-об'єкти використовується `RowMapper<T>` :

- Функціональний інтерфейс `RowMapper<T>` містить метод `T mapRow(ResultSet rs, int rowNum)` , що дозволяє описувати мапінг через лямбда-вирази.
- Застосування незмінюваних записів (Java records) у парі з `RowMapper` створює компактні та надійні структури даних.
- Метод `query()` виконує вибірку списку сутностей, а `queryForObject()` очікує повернення рівно одного рядка.

Приклад реалізації RowMapper для Java Record

```
RowMapper<CameraRecord> cameraRowMapper = (rs, rowNum) -> new CameraRecord(  
    rs.getLong("id"),  
    rs.getString("name"),  
    rs.getString("ip_address"),  
    rs.getString("model")  
);
```

- Конструктор запису приймає значення безпосередньо зі стовпців `ResultSet` .
- Строга типізація вибірки запобігає помилкам приведення типів у сервісах.

Читання даних через репозиторій на базі JdbcTemplate

```
@Repository
public class JdbcCameraRepository {
    private final JdbcTemplate jdbcTemplate;

    public JdbcCameraRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public List<CameraRecord> findAll() {
        var sql = "SELECT id, name, ip_address, model FROM cameras";
        return jdbcTemplate.query(sql, (rs, rowNum) -> new CameraRecord(
            rs.getLong("id"), rs.getString("name"), rs.getString("ip_address"), rs.getString("model")
        ));
    }
}
```

Модифікація даних через репозиторій на базі JdbcTemplate

```
@Repository
public class JdbcCameraRepository {

    private final JdbcTemplate jdbcTemplate;

    public JdbcCameraRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public void insert(CameraRecord camera) {
        var sql = "INSERT INTO cameras (name, ip_address, model) VALUES (?, ?, ?)";
        jdbcTemplate.update(sql, camera.name(), camera.ipAddress(), camera.model());
    }
}
```

Сучасний підхід до SQL через JdbcClient

- Інтерфейс `JdbcClient` пропонує зручний явний (fluent) API для побудови та виконання SQL-запитів.
- Автоматично налаштовується у Spring Boot та впроваджується через конструктор репозиторію.
- Забезпечує гнучкість нативного SQL із безпечною передачею іменованих або позиційних параметрів.

Автоматичний мапінг записів у JdbcClient

```
@Repository
public class ClientCameraRepository {
    private final JdbcClient jdbcClient;

    public ClientCameraRepository(JdbcClient jdbcClient) {
        this.jdbcClient = jdbcClient;
    }

    public List<CameraRecord> findByName(String query) {
        return jdbcClient.sql("SELECT id, name, ip_address, model FROM cameras WHERE name LIKE :name")
            .param("name", "%" + query + "%")
            .query(CameraRecord.class)
            .list();
    }
}
```

Обмеження прямого доступу до бази даних через SQL

Попри зручність `JdbcTemplate` та `JdbcClient`, прямий SQL має суттєві обмеження:

- Рядкові SQL-запити перевіряються лише під час виконання програми, що унеможливлює пошук синтаксичних помилок на етапі компіляції.
- Моделювання зв'язків між таблицями вимагає громіздких SQL JOIN та ручної збірки графів об'єктів.
- Відсутність автоматичного відстеження змін вимагає явного виклику операцій оновлення для кожного об'єкта.

Розрив між реляційною та об'єктною парадигмами

При роботі з реляційними базами даних у Java виникає розбіжність моделей:

- Реляційна модель базується на таблицях, рядках, ключах та нормалізації даних.
- Об'єктно-орієнтована модель оперує об'єктами, графами зв'язків, наслідуванням та поліморфізмом.
- Ця розбіжність відома як об'єктно-реляційний розрив (Object-Relational Impedance Mismatch).

Призначення технології Object-Relational Mapping

Для подолання об'єктно-реляційного розриву застосовується технологія ORM:

- Навіть з використанням `JdbcTemplate` розробник змушений вручну писати мапінг між таблицями та об'єктами.
- Технологія **ORM (Object-Relational Mapping)** автоматизує перетворення рядків таблиць реляційної БД у повноцінні Java-об'єкти.
- ORM ізолює бізнес-код від низькорівневих деталей виконання запитів.

Проблема об'єктно-реляційного розриву

Як поєднати багату об'єктну модель (ієрархії класів, колекції зв'язків) із плоскими таблицями реляційної бази без написання тисяч рядків ручного SQL мапінгу?

Які стандарти гарантують незалежність бізнес-коду від конкретного постачальника бази даних (PostgreSQL, Oracle чи H2)?

Специфікація Jakarta Persistence та провайдер Hibernate

Стандарт Jakarta Persistence (JPA) визначає правила об'єктно-реляційного відображення:

- Специфікація JPA виступає контрактом і містить анотації в пакеті `jakarta.persistence.*` (`@Entity` , `@Id` , `EntityManager`).
- Hibernate є провайдером і реалізацією JPA, виконуючи генерацію SQL, керування кешем та транзакціями.
- Spring Data JPA є надбудовою, що спрощує використання `EntityManager` через інтерфейси репозиторіїв.

Багатошарова архітектура доступу до даних у Spring Boot

```
Spring Data JPA (Інтерфейси репозиторіїв)  
└─> Jakarta Persistence API (Специфікація JPA 3.2)  
    └─> Hibernate ORM 6.x (Провайдер реалізації)  
        └─> JDBC Driver ─> База даних (PostgreSQL / H2)
```

Оголошення базової сутності JPA

```
@Entity
@Table(name = "cameras")
public class Camera {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(name = "ip_address", unique = true, nullable = false, length = 45)
    private String ipAddress;

    @Column(length = 100)
    private String model;
}
```

Конструктори та інкапсуляція в сутностях JPA

```
public class Camera {  
    // поля сутності  
  
    protected Camera() {  
        // Обов'язковий no-arg конструктор для механізмів рефлексії JPA  
    }  
  
    public Camera(String name, String ipAddress, String model) {  
        this.name = name;  
        this.ipAddress = ipAddress;  
        this.model = model;  
    }  
  
    public Long getId() { return id; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

Стратегії генерації первинних ключів

Анотація `@GeneratedValue` визначає спосіб формування ідентифікатора:

Стратегія	Механізм формування	Особливості
IDENTITY	<code>AUTO_INCREMENT</code> стовпець БД	Блокує пакетну вставку (JDBC batching).
SEQUENCE	Об'єкт <code>SEQUENCE</code> у базі даних	Підтримує batching через пул ID (<code>allocationSize</code>).
UUID	Формування 128-бітного значення	Ефективна для розподілених систем.
AUTO	Вибір стратегії провайдером JPA	Не рекомендується через непередбачуваність.

Приклад використання анотації GeneratedValue

```
@Entity
public class Camera {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "camera_seq")
    @SequenceGenerator(name = "camera_seq", sequenceName = "cameras_seq")
    private Long id;
}

@Entity
public class UserAccount {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
}
```

Обмеження незмінності та рефлексії Java Records

Спроба оголосити сутність як `record Camera(...)` є антипатерном:

- Незмінність типу Record забороняє модифікацію полів після створення об'єкта.
- Відсутність конструктора без аргументів унеможлиблює створення екземпляра через рефлексію під час читання з бази.

Обмеження проксіювання та фінальності класів Record

Архітектурні перешкоди для використання Records як Entities:

- Фінальність класів Record унеможливорює створення динамічних підкласів-проксі бібліотекою ByteBuddy.
- Без динамічних проксі Hibernate не здатен забезпечити ліниве завантаження зв'язаних даних.
- Сутності JPA завжди реалізуються як традиційні класи з гетерами та сетерами.

Принципова різниця між Entity, DTO та моделлю сервісу

Кожен шар архітектури вирішує власну задачу та оперує відповідною моделлю:

- Сутність JPA відображає схему таблиці в реляційній базі даних і має змінюваний стан для підтримки механізму автоматичного відстеження змін Hibernate.
- Об'єкт передачі даних (DTO) виступає стабільним контрактом взаємодії з клієнтами та запобігає витоку внутрішньої структури бази даних або циклічним посиланням під час серіалізації.
- Окремий доменний клас сервісного шару інкапсулює бізнес-правила системи та залишається повністю чистим від сторонніх бібліотечних анотацій JPA чи JSON.

Способи перетворення даних між шарами застосунку

Для перенесення стану між моделями використовують ручний мапінг або спеціалізовані бібліотеки:

- Ручне перетворення через статичні фабричні методи чи окремі компоненти мапінгу надає повний контроль над перенесенням полів без сторонніх залежностей.
- Інструменти генерації коду під час компіляції (зокрема [MapStruct](#)) автоматично створюють швидкі реалізації інтерфейсів мапінгу з перевіркою типів без накладних витрат у runtime.
- Використання динамічних бібліотек на основі рефлексії (наприклад `ModelMapper`) не рекомендується через суттєве уповільнення роботи та складність пошуку помилок.

Основні типи кардинальності зв'язків

Специфікація JPA підтримує стандартні типи зв'язків між сутностями:

- Анотація `@ManyToOne` описує зв'язок, коли багато дочірніх записів посилаються на один батьківський (наприклад, камери та їхня локація).
- Анотація `@OneToMany` представляє дзеркальну сторону зв'язку, де батьківський запис містить колекцію дочірніх сутностей.
- Найкращою практикою є моделювання зв'язків у вигляді двостороннього зв'язку `@ManyToOne` та `@OneToMany`.

Зв'язки Many-to-Many та One-to-One

Додаткові типи кардинальності у реляційній моделі:

- Анотація `@ManyToMany` описує зв'язок багатьох сутностей з багатьма іншими через окрему проміжну таблицю.
- Анотація `@OneToOne` пов'язує рівно один екземпляр однієї сутності з одним екземпляром іншої.
- Для зв'язків `@OneToOne` та `@ManyToOne` обов'язково встановлюється ліниве завантаження `FetchType.LAZY`.

Визначення сторони-власника реляційного зв'язку

У двосторонньому зв'язку обов'язковим є визначення сторони-власника:

- Сторона-власник містить анотацію `@JoinColumn` та фізичний стовпець зовнішнього ключа в таблиці бази даних.
- Зворотна сторона обов'язково вказує атрибут `mappedBy` із назвою відповідного поля в сутності-власнику.
- Якщо пропустити атрибут `mappedBy`, Hibernate помилково створить зайву проміжну таблицю зв'язку.

Приклад оголошення зворотного боку зв'язку

```
@Entity
public class Location {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Зворотна сторона: mappedBy вказує на поле 'location' у класі Camera
    @OneToMany(mappedBy = "location", cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<Camera> cameras = new HashSet<>();
}
```

Стратегії каскадування операцій життєвого циклу

Для узгодженого управління зв'язаними сутностями застосовується каскадування:

- Типи `CascadeType.PERSIST` та `MERGE` забезпечують автоматичне збереження або оновлення дочірніх об'єктів разом із батьківським.
- Значення `CascadeType.ALL` поширює всі операції життєвого циклу, включно з видаленням, на дочірні сутності.
- Каскадування спрощує збереження складних агрегатів без виклику окремих методів репозиторію.

Механізм видалення сиріт orphanRemoval

```
public void removeCamera(Camera camera) {  
    cameras.remove(camera);  
    camera.setLocation(null);  
}
```

- Прапорець `orphanRemoval = true` генерує команду `DELETE` для рядка дочірньої сутності після її вилучення з колекції.
- `CascadeType.REMOVE` видаляє дітей лише при видаленні батька, тоді як `orphanRemoval` реагує на розрив посилання в пам'яті.

Оголошення зв'язку Many-to-Many через проміжну таблицю

```
@Entity
public class Camera {
    @ManyToMany
    @JoinTable(
        name = "camera_tags_mapping",
        joinColumns = @JoinColumn(name = "camera_id"),
        inverseJoinColumns = @JoinColumn(name = "tag_id")
    )
    private Set<CameraTag> tags = new HashSet<>();
}
```

Правила реалізації контрактів equals та hashCode

Для JPA-сутностей стандартна генерація `equals` та `hashCode` не підходить:

- Порівняння за автоінкрементним `id` є хибним, оскільки до збереження в базу значення ідентифікатора дорівнює `null`.
- Генерація за всіма полями змінює значення `hashCode` при модифікації стану сутності, що ламає колекції `HashSet` та `HashMap`.
- Рекомендований підхід: порівняння за стабільним бізнес-ключем (наприклад, унікальний `ipAddress`) або перевірка ідентичності класів та рівності збережених `id`.

Відмінності між FetchType EAGER та LAZY

Специфікація JPA визначає дві стратегії завантаження зв'язаних даних:

- Жадібне завантаження `FetchType.EAGER` вичитує пов'язані сутності з бази негайно разом із основним записом.
- Ліниве завантаження `FetchType.LAZY` вичитує дані лише у момент першого фактичного звернення до гетера або колекції.
- Золоте правило високопродуктивної розробки: **Усі зв'язки повинні бути `LAZY`.**

Порівняння стратегій завантаження за замовчуванням

Анотація зв'язку	Стратегія за замовчуванням	Рекомендована стратегія
@OneToMany	FetchType.LAZY	FetchType.LAZY
@ManyToMany	FetchType.LAZY	FetchType.LAZY
@ManyToOne	FetchType.EAGER (Небезпечно)	FetchType.LAZY (Обов'язково)
@OneToOne	FetchType.EAGER (Небезпечно)	FetchType.LAZY (Обов'язково)

Причини виникнення LazyInitializationException

HTTP Request -> Controller -> Service (@Transactional) -> DB

|
Транзакцію закрито

|
Controller / View

|
location.getCameras().size()

|
LazyInitializationException

Небезпека та обмеження антипатерну Open Session in View

За замовчуванням у Spring Boot увімкнено режим Open Session in View (OSIV):

- Параметр `spring.jpa.open-in-view=true` утримує сесію бази даних відкритою протягом усього часу обробки HTTP-запиту, включно з рендерингом відповіді.
- Це маскує помилки `LazyInitializationException`, але призводить до вичерпання пулу з'єднань через утримання з'єднань повільними клієнтами.
- У промислових системах рекомендується вимикати цей режим (`false`), а необхідні зв'язані дані завантажувати в межах сервісної транзакції через DTO або `JOIN FETCH`.

Деградація продуктивності при ітерації по зв'язках

Що відбудеться на рівні SQL-запитів до бази, якщо ми вичитаємо список зі 100 локацій, а потім у циклі звернемося до списку камер кожної локації

```
location.getCameras().size() ?
```

Скільки звернень до бази даних буде виконано через мережевий сокет?

Механіка виникнення проблеми N+1 запиту

Проблема N+1 запитів — це головна причина падіння швидкодії ORM:

1. Виконується 1 первинний запит для вибірки списку локацій:

```
SELECT * FROM locations; -- Повертає N = 100 рядків
```

2. Для кожної зі 100 локацій у циклі генерується N окремих додаткових запитів:

```
SELECT * FROM cameras WHERE location_id = 1;  
SELECT * FROM cameras WHERE location_id = 2;
```

Наслідки проблеми N+1 для продуктивності системи

Деградація ресурсів та хибні методи оптимізації:

- Замість одного запиту з об'єднанням виконується 101 звернення через мережу, що вичерпує пул з'єднань.
- Зміна стратегії з `LAZY` на `EAGER` не вирішує проблему, а лише змушує фреймворк виконувати ті самі 101 запит негайно.
- Для реального вирішення необхідні спеціалізовані запити або графи сутностей.

Принцип роботи директиви JPQL JOIN FETCH

Найпростішим рішенням N+1 є жадібне завантаження через **JOIN FETCH** :

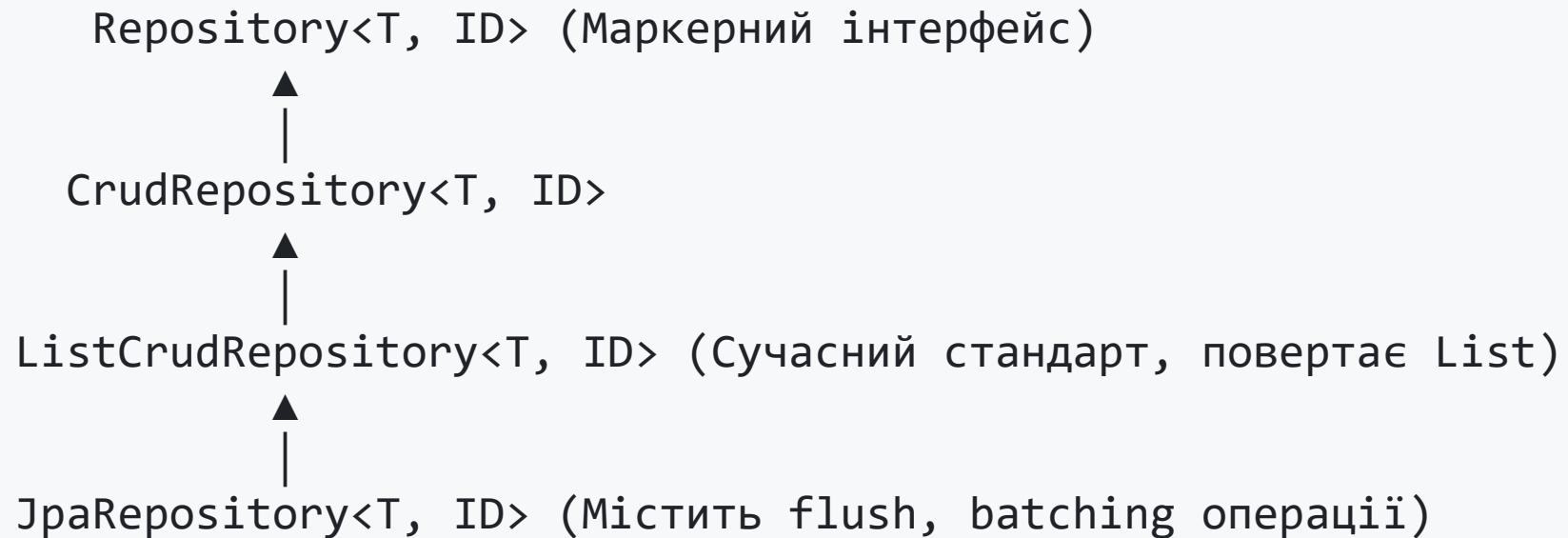
- Директива **JOIN FETCH** вказує Hibernate завантажити пов'язані сутності в одному SQL-запиті через **LEFT JOIN** .
- Ключове слово **DISTINCT** гарантує видалення дублікатів кореневих об'єктів, які виникають через декартів добуток рядків.
- Запит повертає повністю ініціалізовані сутності без потреби подальших звернень до бази.

Реалізація вибірки через JOIN FETCH у репозиторії

```
public interface LocationRepository extends ListCrudRepository<Location, Long> {  
    @Query("SELECT DISTINCT l FROM Location l JOIN FETCH l.cameras")  
    List<Location> findAllWithCameras();  
}
```

- Метод повертає список локацій разом із завантаженими камерами за один мережевий запит.
- Це повністю усуває ризик виникнення `LazyInitializationException` у веб-шарі.

Ієрархія інтерфейсів Spring Data Repository



Оголошення типізованого репозиторію в Spring Boot

```
@Repository
public interface CameraRepository extends ListCrudRepository<Camera, Long> {

    // Spring Data на етапі запуску динамічно створює проксі-реалізацію
}
```

Приклади формування запитів за назвою методу

```
public interface CameraRepository extends ListCrudRepository<Camera, Long> {  
  
    // SELECT * FROM cameras WHERE lower(name) = lower(?) AND status = ?  
    List<Camera> findByNameIgnoreCaseAndStatus(String name, CameraStatus status);  
  
    // SELECT * FROM cameras WHERE ip_address IS NOT NULL ORDER BY name ASC  
    List<Camera> findByIpAddressIsNotNullOrderByNameAsc();  
  
    // SELECT COUNT(*) > 0 FROM cameras WHERE ip_address = ?  
    boolean existsByIpAddress(String ipAddress);  
}
```

Ключові слова синтаксичного аналізатора Spring Data

Синтаксичний аналізатор підтримує широкий набір конструкцій:

- Логічні оператори: `And` , `Or` .
- Оператори порівняння: `Between` , `LessThan` , `GreaterThan` , `Like` , `StartingWith` .
- Робота з регістром та сортуванням: `IgnoreCase` , `OrderBy...Asc` , `OrderBy...Desc` .
- Перевірка існування та кількості: `existsBy...` , `countBy...` .

Особливості використання мови запитів JPQL

Коли запит занадто складний для імені методу, застосовують `@Query` :

- Мова JPQL оперує назвами Java-сутностей та їх полів, а не іменами фізичних таблиць.
- Запити JPQL є переносними між різними СУБД, оскільки Hibernate транслює їх у діалект цільової бази даних.
- Забезпечується захист від помилок у назвах полів під час валідації запитів на етапі запуску програми.

Приклад параметризованого запиту JPQL у репозиторії

```
public interface CameraRepository extends ListCrudRepository<Camera, Long> {  
  
    @Query("SELECT c FROM Camera c WHERE c.status = :status AND c.location.id = :locationId")  
    List<Camera> findActiveByLocation(  
        @Param("status") CameraStatus status,  
        @Param("locationId") Long locationId  
    );  
}
```

- Іменовані параметри прив'язуються за допомогою анотації `@Param`.

Приклад використання нативного SQL-запиту

```
public interface CameraRepository extends ListCrudRepository<Camera, Long> {  
  
    @Query(value = ""  
        SELECT * FROM cameras  
        WHERE status = 'ACTIVE'  
        AND ip_address LIKE :subnet  
        """, nativeQuery = true)  
    List<Camera> findActiveInSubnet(@Param("subnet") String subnet);  
}
```

Особливості та ризики нативних запитів

Специфіка роботи з прямими SQL-командами:

- Параметр `nativeQuery = true` вмикає парсер JPQL і передає рядок без змін у JDBC драйвер.
- Дозволяє використовувати специфічні функції діалекту, зокрема роботу з JSONB або аналітичні функції.
- Вимагає підвищеної уваги під час міграцій.

Робота з In-Memory базами даних H2

Для локальної розробки, прототипування та швидкого запуску інтеграційних тестів використовується вбудована база даних H2:

- Повністю написана на Java, запускається безпосередньо у просторі пам'яті JVM процесу додатку.
- Не потребує встановлення та запуску окремого сервера СУБД.
- Очищує свій стан після завершення роботи застосунку.

```
<dependency>  
  <groupId>com.h2database</groupId>  
  <artifactId>h2</artifactId>  
  <scope>runtime</scope>  
</dependency>
```

Конфігурація підключення до H2 в application.yml

```
spring:
  datasource:
    url: jdbc:h2:mem:sampledb;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

- Параметр `MODE=PostgreSQL` емулює діалектні особливості цільової бази даних.

Активація та використання веб-консолі H2

```
spring:  
  h2:  
    console:  
      enabled: true  
      path: /h2-console  
  jpa:  
    properties:  
      hibernate:  
        format_sql: true
```

- Параметр `spring.h2.console.enabled` активує вбудовану панель за адресою `/h2-console`.
- Опція `format_sql: true` робить вивід SQL у журналах структурованим та зручним для читання.

Висновки

1. Інструменти `JdbcTemplate` та `JdbcClient` забезпечують прямий контроль над SQL, а Jakarta Persistence автоматизує об'єктне відображення.
2. Сторона-власник реляційного зв'язку визначається зовнішнім ключем, а зворотна сторона обов'язково маркується атрибутом `mappedBy`.
3. Проблема N+1 запитів є критичним ризиком продуктивності та надійно вирішується за допомогою `JOIN FETCH`.
4. Бібліотека Spring Data JPA суттєво скорочує рутинний код завдяки автоматичній генерації запитів за назвою методу.

Індивідуальне завдання

Вимоги до конфігурації проєкту та моделі даних:

1. Підключити базу даних H2 та додати стартер `spring-boot-starter-jdbc` у конфігурацію збірки проєкту.
2. Описати модель даних предметної області як незмінюваний запис Java 25 (`record`).

Індивідуальне завдання

Вимоги до репозиторію та сервісного шару:

1. Реалізувати репозиторій із впровадженням компонента `JdbcClient`.
2. Реалізувати вибірку списку сутностей та пошук за ідентифікатором із мапінгом у Java Record через `.query(Record.class)`.
3. Реалізувати операцію додавання нового запису через `.sql().param().update()`.
4. Оголосити REST-контролер для взаємодії з клієнтами.

(Результати завантажити на distedu до дедлайну)

Індивідуальне завдання

Порядок перевірки створення та читання даних:

1. Запустити застосунок через `./mvnw spring-boot:run` або `./gradlew bootRun` та переконатися в успішному старті сервісу.

2. Надіслати POST-запит для створення нового запису:

```
curl -X POST http://localhost:8080/api/cameras -H "Content-Type: application/json" -d "{\"name\":\"Камера\",\"model\":\"PTZ-01\"}"
```

3. Надіслати GET-запит для перевірки отримання списку:

```
curl http://localhost:8080/api/cameras
```

Індивідуальне завдання

Порядок перевірки фільтрації та веб-консолі H2:

1. Надіслати GET-запит із параметром пошуку за назвою через ендпоінт фільтрації:

```
curl "http://localhost:8080/api/cameras/search?name=Камера"
```

2. Увімкнути веб-консоль H2 у файлі `application.properties` та відкрити адресу `http://localhost:8080/h2-console`.

3. Виконати SQL-запит `SELECT * FROM cameras` у консолі H2 та перевірити збереження доданих записів.

Групове завдання

Вимоги до схеми, CRUD-операцій та оптимізації вибірок:

1. Спроектувати реляційну схему командного проєкту, що містить щонайменше 4 пов'язані сутності з різними типами відношень.
2. Реалізувати повноцінний набір операцій CRUD для кожної з 4 сутностей через REST-контролери, сервісний шар та репозиторії.
3. Налаштувати оптимізовані вибірки колекцій через `JOIN FETCH` для повного усунення проблеми N+1 запитів.

Групове завдання

Вимоги до цілісності даних та операцій репозиторіїв:

1. Визначити правила каскадування операцій через `CascadeType` та підключити `orphanRemoval = true` для дочірніх сутностей.
2. Оголосити методи пошуку в репозиторіях за конвенцією імен та кастомні вибірки через `@Query`.

Групове завдання

Критерії та інструкція перевірки шару збереження даних:

1. Успішний запуск застосунку та виконання базових перевірок через виклик `./mvnw test` або `./gradlew test` без помилок.
2. Верифікація відсутності проблеми N+1 у журналі SQL-запитів (`spring.jpa.show-sql=true`) із підтвердженням виконання єдиного запиту з `LEFT JOIN`.
3. Перевірка ендпоінтів через HTTP-запит `curl` `"http://localhost:8080/api/cameras"` із валідацією коректності отриманого списку DTO.